

04_computationalTime

August 31, 2026

```
[1]: import re
import pandas as pd
import os
import glob
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"

import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(
    style="whitegrid",
    font="DejaVu Sans",    # change to Helvetica or DejaVu Sans depending on OS
    rc={
        "axes.titlesize": 14,
        "axes.labelsize": 12,
        "xtick.labelsize": 10,
        "ytick.labelsize": 10,
        "legend.fontsize": 10
    }
)
```

```
[2]: import csv
from itertools import product
import pandas as pd

def extract_metrics(filepath):
    with open(filepath, newline="") as f:
        row = next(csv.DictReader(f, delimiter="\t"))
    return {
        "time_s": float(row["s"]),
        "max_rss_mb": float(row["max_rss"]),
        "cpu_time_s": float(row["cpu_time"]),
    }
```

```

        "mean_load_pct": float(row["mean_load"]),
        "io_in_mb": float(row["io_in"]),
        "io_out_mb": float(row["io_out"]),
    }

def aggregate_long(templates, variables, extra=None):
    """
    templates : dict[tool] -> list of (step_label, filename_template)
    variables : dict[placeholder] -> list of values (cartesian product)
    extra      : dict of constant columns to attach to every row (e.g. {"depth": 1,
    ↪ "full"})

    Returns a long-format list of row-dicts, one per (tool, step, combo),
    with the 4 metrics as columns. Missing/unreadable files are skipped
    with a warning instead of raising.
    """
    extra = extra or {}
    keys = list(variables.keys())
    rows = []
    missing = []

    for combo_values in product(*variables.values()):
        combo = dict(zip(keys, combo_values))
        for tool, steps in templates.items():
            for step_label, tpl in steps:
                filepath = benchmarks_path + tpl.format(**combo)

                if not os.path.exists(filepath):
                    missing.append(filepath)
                    continue

                try:
                    metrics = extract_metrics(filepath)
                except (StopIteration, KeyError, ValueError) as e:
                    warnings.warn(f"Skipping unreadable file {filepath}: {e}")
                    missing.append(filepath)
                    continue

                rows.append({
                    **extra,
                    **combo,
                    "tool": tool,
                    "step": step_label,
                    "file": filepath,
                    **metrics,
                })

```

```

if missing:
    print(f"\n{len(missing)} file(s) missing or unreadable, skipped:")
    for m in missing:
        print(f" {m}")

return rows

# to plot real data

def plot_stacked_metric_by(df, metric, group_col, title=None, ax=None,
    ylim=None,
                            group_order=None, filter_query=None,
                            step_colors=None):
    sub = df.query(filter_query).copy() if filter_query else df.copy()

    if group_order is not None:
        sub[group_col] = pd.Categorical(sub[group_col], categories=group_order,
    ordered=True)

    pivot = sub.pivot_table(
        index=["tool", group_col], columns="step", values=metric,
        aggfunc="sum", fill_value=0
    )
    pivot = pivot.sort_index(level=["tool", group_col])

    if step_colors is not None:
        ordered_steps = [s for s in step_colors if s in pivot.columns]
        ordered_steps += [s for s in pivot.columns if s not in step_colors]
        pivot = pivot[ordered_steps]
        colors = [step_colors.get(s, "#999999") for s in ordered_steps]
    else:
        colors = None

    if ax is None:
        fig, ax = plt.subplots(figsize=(12, 10))

    pivot.plot(kind="bar", stacked=True, ax=ax, color=colors, edgecolor="none",)
    ax.set_ylabel(metric)
    ax.set_xlabel(f"tool, {group_col}")
    ax.set_title(title or f"{metric} by {group_col}")
    if ylim is not None:
        ax.set_ylim(*ylim)
    ax.legend(title="step", bbox_to_anchor=(1.02, 1), loc="upper left")
    plt.setp(ax.get_xticklabels(), rotation=45, ha="right")
    return ax

```

```

def plot_grouped_metric_by(df, metric, group_col, title=None, ax=None,
    ylim=None,
                                group_order=None, filter_query=None,
                                step_colors=None, aggfunc="max", width=0.9):
    sub = df.query(filter_query).copy() if filter_query else df.copy()

    if group_order is not None:
        sub[group_col] = pd.Categorical(sub[group_col], categories=group_order,
    ordered=True)

    pivot = sub.pivot_table(
        index=["tool", group_col], columns="step", values=metric,
        aggfunc=aggfunc, fill_value=0
    )
    pivot = pivot.sort_index(level=["tool", group_col])

    if step_colors is not None:
        ordered_steps = [s for s in step_colors if s in pivot.columns]
        ordered_steps += [s for s in pivot.columns if s not in step_colors]
        pivot = pivot[ordered_steps]
        colors = [step_colors.get(s, "#999999") for s in ordered_steps]
    else:
        colors = None

    if ax is None:
        fig, ax = plt.subplots(figsize=(12, 10))

    pivot.plot(kind="bar", stacked=False, ax=ax, color=colors,
    edgecolor="none", width=width)
    ax.set_ylabel(metric)
    ax.set_xlabel(f"tool, {group_col}")
    ax.set_title(title or f"{metric} (per-step {aggfunc}) by {group_col}")
    if ylim is not None:
        ax.set_ylim(*ylim)
    ax.legend(title="step", bbox_to_anchor=(1.02, 1), loc="upper left")
    plt.setp(ax.get_xticklabels(), rotation=45, ha="right")
    return ax

```

```

[3]: benchmarks_path = "/mnt/TEdata/TEbenchmarking/benchmarks/" # set to snakemake
    benchmarks_folder

```

```

[4]: STEP_COLORS = {
    "STARsolo_align": "#4F4F4F",
    "STARsolo_TE_align": "#f0df93",
    "setup_SoloTE": "#A9D7A1",
    "setup_stellarscope": "#9CA3C2",

```

```

    "run_SoloTE":      "#83BF7A",
    "load":            "#7680AB",
    "pseudobulk":      "#4F5D93",
    "onestep":         "#404B77",
}

```

1 Real datasets

```

[15]: sim_templates_real_mouse = {
    "STARsolo_TE": [
        ("STARsolo_TE_align",      "STARsolo_EM_TE_align_{dataset_id}.txt"),
    ],
    "SoloTE": [
        ("STARsolo_align",      "STARsolo_align_{dataset_id}_best.txt"),
        ("setup_SoloTE",        "setup_SoloTE_mm10.txt"),
        ("run_SoloTE",          "run_SoloTE_{dataset_id}.txt"),
    ],
    "stellarscope": [
        ("STARsolo_align",      "STARsolo_align_{dataset_id}_score.txt"),
        ("setup_stellarscope",   "setup_stellarscope_{dataset_id}.
↳txt"),
        ("load",                "run_stellarscope_load_{dataset_id}.txt"),
        ("pseudobulk",          "run_stellarscope_pseudobulk_{dataset_id}.txt"),
    ]
}

sim_templates_real_human = {
    "STARsolo_TE": [
        ("STARsolo_TE_align",      "STARsolo_EM_TE_align_{dataset_id}.txt"),
    ],
    "SoloTE": [
        ("STARsolo_align",      "STARsolo_align_{dataset_id}_best.txt"),
        ("setup_SoloTE",        "setup_SoloTE_hg38.txt"),
        ("run_SoloTE",          "run_SoloTE_{dataset_id}.txt"),
    ],
    "stellarscope": [
        ("STARsolo_align",      "STARsolo_align_{dataset_id}_score.txt"),
        ("setup_stellarscope",   "setup_stellarscope_{dataset_id}.
↳txt"),
        ("load",                "run_stellarscope_load_{dataset_id}.txt"),
        ("pseudobulk",          "run_stellarscope_pseudobulk_{dataset_id}.txt"),
    ]
}

```

```

[16]: rows_mouse = aggregate_long(
    sim_templates_real_mouse,

```

```

variables={"dataset_id": ["10xMouse_RA_LIF",
                          "10xMouse_PBMC_10k"]},
extra={"species": "mouse"},
)

rows_human = aggregate_long(
    sim_templates_real_human,
    variables={"dataset_id": ["10xPBMC_pbmc8k", "10xHuman_8CLC_DUX4"]},
    extra={"species": "human"},
)

df_real = pd.DataFrame(rows_mouse + rows_human)
#df_real.to_csv("runtime_summary_real.csv", index=False)

```

```

[17]: dataset_ids = ["10xMouse_RA_LIF", "10xMouse_PBMC_10k", "10xHuman_8CLC_DUX4",
                    ↪ "10xPBMC_pbmc8k"]

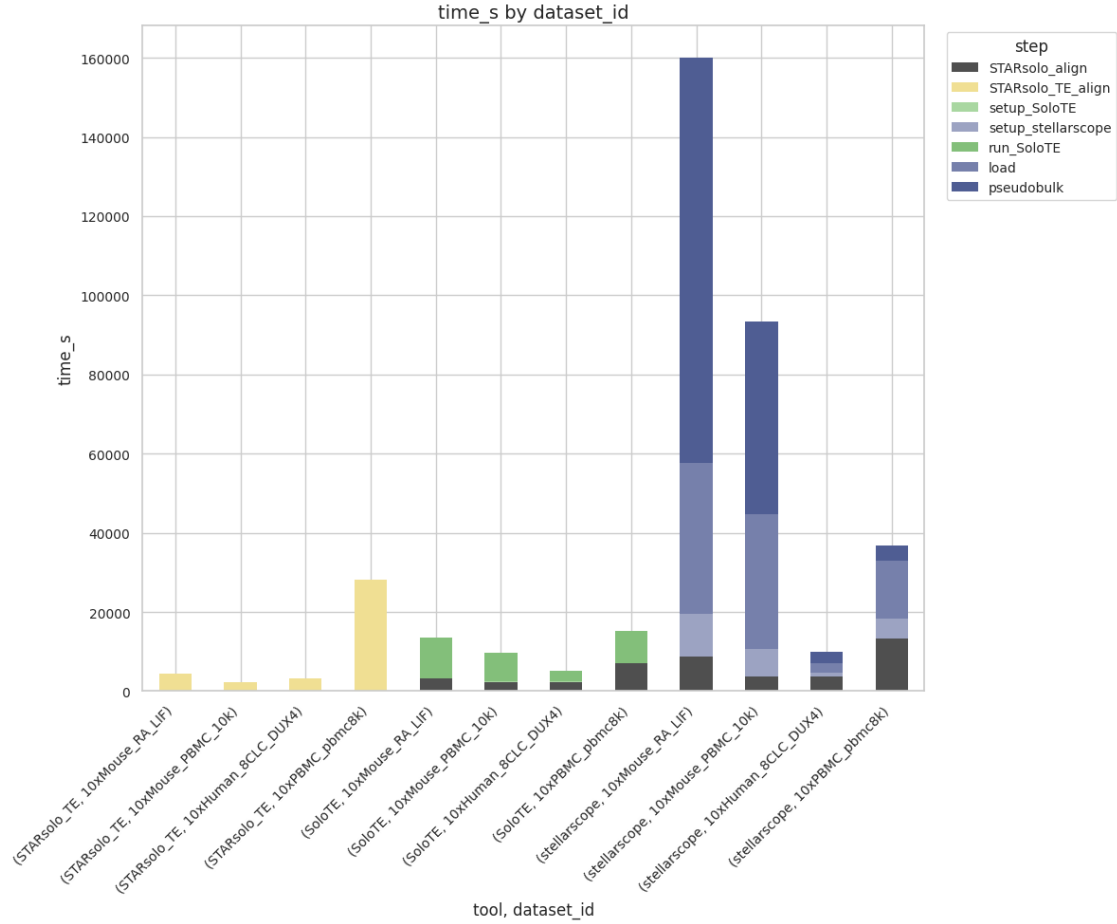
plot_stacked_metric_by(
    df_real,
    metric="time_s",
    group_col="dataset_id",
    group_order=dataset_ids,           # controls x-axis order
    step_colors=STEP_COLORS,
    filter_query=f"dataset_id in {dataset_ids}", # restricts to just this list
)
plt.tight_layout()
plt.savefig("figures/benchmarking_time_realddata_stacked.pdf", dpi=300)
plt.show()

```

```

[17]: <Axes: title={'center': 'time_s by dataset_id'}, xlabel='tool, dataset_id',
      ylabel='time_s'>

```



2 Simulations

```
[23]: sim_templates_depths = {
    "STARsolo_TE": [
        ("STARsolo_TE_align", □
↪ "STARsolo_EM_TE_align_simulated_mm_RA_{depth}_{age}.txt"),
    ],
    "SoloTE": [
        ("STARsolo_align", "STARsolo_align_simulated_mm_RA_{depth}_{age}_best.
↪ txt"),
        ("setup_SoloTE", "setup_SoloTE_mm10.txt"),
        ("run_SoloTE", "run_SoloTE_simulated_mm_RA_{depth}_{age}.txt"),
    ],
    "stellarscope": [
        ("STARsolo_align", □
↪ "STARsolo_align_simulated_mm_RA_{depth}_{age}_score.txt"),
        ("setup_stellarscope", "setup_stellarscope_simulated_mm_RA_{age}.txt"),
    ]
}
```

```

        ("load",
        ↪"run_stellarscope_load_simulated_mm_RA_{depth}_{age}.txt"),
        ("pseudobulk",
        ↪"run_stellarscope_pseudobulk_simulated_mm_RA_{depth}_{age}.txt"),
    ],
    "stellarscope_onestep": [
        ("STARsolo_align",
        ↪"STARsolo_align_simulated_mm_RA_{depth}_{age}_score.txt"),
        ("setup_stellarscope", "setup_stellarscope_simulated_mm_RA_{age}.txt"),
        ("onestep",
        ↪"run_stellarscope_onestep_simulated_mm_RA_{depth}_{age}.txt"),
    ],
}

sim_templates_full = {
    "STARsolo_TE": [
        ("STARsolo_TE_align", "STARsolo_EM_TE_align_simulated_mm_RA_{age}.txt"),
    ],
    "SoloTE": [
        ("STARsolo_align", "STARsolo_align_simulated_mm_RA_{age}_best.txt"),
        ("setup_SoloTE", "setup_SoloTE_mm10.txt"),
        ("run_SoloTE", "run_SoloTE_simulated_mm_RA_{age}.txt"),
    ],
    "stellarscope": [
        ("STARsolo_align", "STARsolo_align_simulated_mm_RA_{age}_score.
        ↪txt"),
        ("setup_stellarscope", "setup_stellarscope_simulated_mm_RA_{age}.txt"),
        ("load", "run_stellarscope_load_simulated_mm_RA_{age}.
        ↪txt"),
        ("pseudobulk",
        ↪"run_stellarscope_pseudobulk_simulated_mm_RA_{age}.txt"),
    ],
    "stellarscope_onestep": [
        ("STARsolo_align", "STARsolo_align_simulated_mm_RA_{age}_score.
        ↪txt"),
        ("setup_stellarscope", "setup_stellarscope_simulated_mm_RA_{age}.txt"),
        ("onestep", "run_stellarscope_onestep_simulated_mm_RA_{age}.
        ↪txt"),
    ],
}

```

```

[24]: rows_depths = aggregate_long(
    sim_templates_depths,
    variables={
        "depth": ["5kRpC", "20kRpC", "50kRpC"],
        "age": ["young", "old"],
    },

```



```

    },
)

rows_full = aggregate_long(
    sim_templates_full,
    variables={"age": ["young", "old"]},
    extra={"depth": "full"},
)

df_sim = pd.DataFrame(rows_depths + rows_full)
print(df_sim.head())
df_sim.shape

```

1 file(s) missing or unreadable, skipped:

/mnt/TEdata/TEbenchmarking/benchmarks/run_stellarscope_onestep_simulated_mm_RA
_young.txt

	depth	age	tool	step
0	5kRpC	young	STARsolo_TE	STARsolo_TE_align \
1	5kRpC	young	SoloTE	STARsolo_align
2	5kRpC	young	SoloTE	setup_SoloTE
3	5kRpC	young	SoloTE	run_SoloTE
4	5kRpC	young	stellarscope	STARsolo_align

	file	time_s	max_rss_mb
0	/mnt/TEdata/TEbenchmarking/benchmarks/STARsolo...	233.4112	34230.88 \
1	/mnt/TEdata/TEbenchmarking/benchmarks/STARsolo...	155.8341	30409.54
2	/mnt/TEdata/TEbenchmarking/benchmarks/setup_So...	54.7464	4123.96
3	/mnt/TEdata/TEbenchmarking/benchmarks/run_Solo...	79.6478	223.60
4	/mnt/TEdata/TEbenchmarking/benchmarks/STARsolo...	269.1671	30608.94

	cpu_time_s	mean_load_pct	io_in_mb	io_out_mb
0	651.11	278.78	92.14	9306.28
1	452.72	290.33	0.00	103.14
2	32.19	58.69	62.53	155.68
3	33.44	19.65	428.34	603.97
4	454.86	168.92	3068.62	8418.44

[24]: (87, 11)

```

[25]: manually_replaced_rows = (
    (df_sim.tool == "stellarscope") &
    (df_sim.age == "young") &
    (df_sim.depth == "full") &
    (df_sim.step.isin(["load", "pseudobulk"]))
)
manually_replaced_rows.sum()

```

```

# Get the index labels of the 2 target rows
indices_to_drop = df_sim[manually_replaced_rows].index
indices_to_drop

# Drop them directly from df_sim
df_sim = df_sim.drop(index=indices_to_drop)

```

[25]: 2

[25]: Index([72, 73], dtype='int64')

```

[26]: manually_replaced_rows2 = (
        (df_sim.tool == "stellarscope_onestep") &
        (df_sim.age == "young") &
        (df_sim.depth == "full") &
        (df_sim.step.isin(["onestep"])))
    )
manually_replaced_rows2.sum()
# This keeps everything EXCEPT those bad rows
df_sim = df_sim.loc[~manually_replaced_rows2, :].copy()
df_sim.shape

```

[26]: 0

[26]: (85, 11)

```

[27]: df_sim.loc[
        (df_sim.tool == "stellarscope_onestep") &
        (df_sim.age == "young") &
        (df_sim.depth == "full") , :
    ]

```

```

[27]:   depth  age      tool      step
74  full  young  stellarscope_onestep  STARSolo_align \
75  full  young  stellarscope_onestep  setup_stellarscope

```

	file	time_s	max_rss_mb
74	/mnt/TEdata/TEbenchmarking/benchmarks/STARSolo...	4852.8070	30608.91 \
75	/mnt/TEdata/TEbenchmarking/benchmarks/setup_st...	6673.6544	9828.28

	cpu_time_s	mean_load_pct	io_in_mb	io_out_mb
74	15297.38	315.22	28466.84	379885.01
75	10969.95	228.43	38465.36	83799.96

```

[28]: # taken from manual runs on VM with more RAM
manual_rows = [

```

```

{
    "tool": "stellarscope",
    "step": "load",
    "depth": "full",
    "age": "young",
    "file": "manual_entry",
    "time_s": 83762.0,
    "max_rss_mb": None,
    "cpu_time_s": None,
    "mean_load_pct": None,
},
{
    "tool": "stellarscope",
    "step": "pseudobulk",
    "depth": "full",
    "age": "young",
    "file": "manual_entry",
    "time_s": 415642,
    "max_rss_mb": None,
    "cpu_time_s": None,
    "mean_load_pct": None,
},
{
    "tool": "stellarscope_onestep",
    "step": "onestep",
    "depth": "full",
    "age": "young",
    "file": "manual_entry",
    "time_s": 536841,
    "max_rss_mb": None,
    "cpu_time_s": None,
    "mean_load_pct": None,
}
]

df_manual = pd.DataFrame(manual_rows)
df_sim = pd.concat([df_sim, df_manual], ignore_index=True)

```

```

[7]: import matplotlib.pyplot as plt
import pandas as pd

def plot_stacked_faceted(df, metric, facet_col, group_col, tools=None,
                        step_colors=None, group_order=None,
                        filter_query=None, sharey=False,
                        figsize_per_tool=(4, 5), width=0.7):
    """

```

One subplot per tool, x-axis = group_col, stacked by step, on `metric`.

```
facet_col    : column to facet by (here: "tool")
group_col    : x-axis grouping within each facet (here: "depth")
tools        : optional list to control facet order/subset
group_order  : optional list to control x-axis category order
filter_query : optional pandas query string, e.g. "age == 'young'"
sharey       : False lets each tool have its own y-scale (recommended
               here given the huge magnitude differences between tools)
"""
sub = df.query(filter_query).copy() if filter_query else df.copy()

if tools is None:
    tools = sub[facet_col].unique().tolist()

if group_order is not None:
    sub[group_col] = pd.Categorical(sub[group_col], categories=group_order,
↳ordered=True)

fig, axes = plt.subplots(
    1, len(tools),
    figsize=(figsize_per_tool[0] * len(tools), figsize_per_tool[1]),
    sharey=sharey,
)
if len(tools) == 1:
    axes = [axes]

for ax, tool in zip(axes, tools):
    tool_sub = sub[sub[facet_col] == tool]

    pivot = tool_sub.pivot_table(
        index=group_col, columns="step", values=metric,
        aggfunc="sum", fill_value=0
    )
    pivot = pivot.loc[:, (pivot != 0).any(axis=0)] # drop all-zero steps
    pivot = pivot.sort_index()

    if step_colors is not None:
        ordered_steps = [s for s in step_colors if s in pivot.columns]
        ordered_steps += [s for s in pivot.columns if s not in step_colors]
        pivot = pivot[ordered_steps]
        colors = [step_colors.get(s, "#999999") for s in ordered_steps]
    else:
        colors = None

    pivot.plot(kind="bar", stacked=True, ax=ax, color=colors,
               edgecolor="none", width=width, legend=False)
```

```

ax.set_title(tool)
ax.set_xlabel("")
plt.setp(ax.get_xticklabels(), rotation=45, ha="right")

axes[0].set_ylabel(metric)

all_handles, all_labels = {}, []
for ax in axes:
    h, l = ax.get_legend_handles_labels()
    for hh, ll in zip(h, l):
        if ll not in all_handles:
            all_handles[ll] = hh
fig.legend(all_handles.values(), all_handles.keys(), title="step",
          bbox_to_anchor=(1.02, 0.9), loc="upper left")

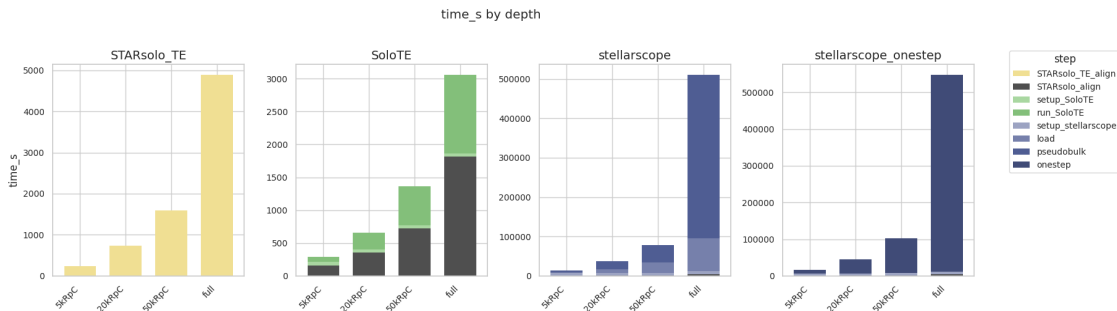
fig.suptitle(f"{metric} by {group_col}", y=1.02)
plt.tight_layout()
return fig, axes

```

```

[8]: fig, axes = plot_stacked_faceted(
    df_sim,
    metric="time_s",
    facet_col="tool",
    group_col="depth",
    tools=["STARsolo_TE", "SoloTE", "stellarscope", "stellarscope_onestep"],
    group_order=["5kRpC", "20kRpC", "50kRpC", "full"],
    filter_query="age == 'young'",
    step_colors=STEP_COLORS,
    sharey=False,
)
plt.savefig("figures/benchmarking_time_sim_faceted_young.pdf",
           bbox_inches="tight")
plt.show()

```



```
[9]: fig, axes = plot_stacked_faceted(
    df_sim,
    metric="time_s",
    facet_col="tool",
    group_col="depth",
    tools=["STARsolo_TE", "SoloTE", "stellarscope", "stellarscope_onestep"],
    group_order=["5kRpC", "20kRpC", "50kRpC", "full"],
    filter_query="age == 'old'",
    step_colors=STEP_COLORS,
    sharey=False,
)
plt.savefig("figures/benchmarking_time_sim_faceted_old.pdf",
           bbox_inches="tight")
plt.show()
```

